

Messaging System Database Architecture (Billions of Messages)

Table of Contents

1. [Overview](#)
 2. [Requirements](#)
 3. [Capacity Estimation](#)
 4. [Schema Design](#)
 5. [ASCII ER Diagram](#)
 6. [Indexing Strategy](#)
 7. [Query Patterns](#)
 8. [Scaling Strategy](#)
 9. [Tradeoffs](#)
 10. [Interview Discussion Points](#)
 11. [Common Interview Follow-ups](#)
 12. [Performance Considerations](#)
 13. [Cross-References](#)
-

Overview

A messaging system is one of the hardest databases to design because it combines high write throughput (messages sent continuously), high read throughput (inbox polling / LISTEN/NOTIFY), ordering guarantees (messages must appear in correct sequence), and massive data volume (years of message history). This schema uses PostgreSQL with LISTEN/NOTIFY for real-time delivery, aggressive partitioning for history, and a dedicated hot table for recent messages to keep the common case fast.

Requirements

Functional Requirements

- 1:1 direct messages between users
- Group conversations (up to 1000 participants)
- Message types: text, image, video, audio, file, reaction, system
- Read receipts per message per participant
- Typing indicators
- Message editing and soft deletion
- Reactions (emoji) on messages
- Notifications: push, email, in-app
- User presence (online/offline/away)
- Pinned messages per conversation
- Message search (full-text)
- Threaded replies

Non-Functional Requirements

- 500M+ registered users
- 100B+ messages stored (historical)
- 10M messages sent per minute at peak
- Sub-100ms message delivery for online users via LISTEN/NOTIFY

- 99.99% uptime for message send
- Messages delivered at-least-once; deduplication at recipient
- Data locality: messages stored near sender's region
- GDPR: account deletion removes personal content

Capacity Estimation

Metric	Estimate
Registered users	500M
Daily active users	100M
Messages per day	60B (avg 600 per DAU)
Peak messages per second	1M
Conversations	10B
Group conversations	500M
Avg group size	25 members
Attachments per day	5B

Storage Estimates

Table	Daily Rows	Row Size	Annual Growth
messages	60B	500B	~11TB/day → 4PB/yr
read_receipts	300B (5 per msg avg)	80B	24TB/day
conversations	100M new/day	400B	14TB/yr
conv_participants	500M new/day	100B	18TB/yr
notifications	200M/day	200B	15TB/yr

Critical insight: At this scale, PostgreSQL handles metadata and recent messages; a purpose-built store (Cassandra, DynamoDB, Apache HBase) handles the historical message archive. This document models the PostgreSQL layer with the transition point clearly marked.

Schema Design

```
-- =====
-- EXTENSIONS
-- =====
CREATE EXTENSION IF NOT EXISTS "uuid-ossf";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";
CREATE EXTENSION IF NOT EXISTS "btree_gist";    -- for range exclusion constraints

-- =====
```

```

-- ENUM TYPES
-- =====
CREATE TYPE user_presence AS ENUM ('online', 'away', 'busy', 'offline');
CREATE TYPE message_type AS ENUM (
    'text', 'image', 'video', 'audio', 'file',
    'sticker', 'gif', 'location', 'contact',
    'system', 'deleted', 'call_started', 'call_ended'
);
CREATE TYPE conversation_type AS ENUM ('direct', 'group', 'channel', 'bot');
CREATE TYPE member_role AS ENUM ('owner', 'admin', 'member');
CREATE TYPE notification_type AS ENUM ('message', 'mention', 'reaction', 'call',
'system');
CREATE TYPE notification_channel AS ENUM ('push', 'email', 'in_app', 'sms');
CREATE TYPE delivery_status AS ENUM ('queued', 'sent', 'delivered', 'read',
'failed');

-- =====
-- USERS
-- =====
CREATE TABLE users (
    user_id          UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    username         VARCHAR(100)  NOT NULL,
    display_name     VARCHAR(200)  NOT NULL,
    email            VARCHAR(320),
    phone            VARCHAR(30),
    avatar_url       VARCHAR(500),
    bio              TEXT,
    is_bot           BOOLEAN        NOT NULL DEFAULT FALSE,
    created_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT uq_username UNIQUE (username)
);

-- Presence: updated every 30 seconds by client heartbeat
-- UNLOGGED for performance (presence data is ephemeral)
CREATE UNLOGGED TABLE user_presence (
    user_id          UUID          PRIMARY KEY REFERENCES users(user_id),
    status           user_presence NOT NULL DEFAULT 'offline',
    status_message   VARCHAR(200),
    last_seen_at     TIMESTAMPTZ   NOT NULL DEFAULT now(),
    device_token     VARCHAR(500), -- for push notifications
    updated_at       TIMESTAMPTZ   NOT NULL DEFAULT now()
);

CREATE TABLE user_settings (
    user_id          UUID          PRIMARY KEY REFERENCES users(user_id),
    push_enabled     BOOLEAN        NOT NULL DEFAULT TRUE,
    email_enabled    BOOLEAN        NOT NULL DEFAULT TRUE,
    quiet_hours_start TIME,
    quiet_hours_end  TIME,
    language         VARCHAR(10)   NOT NULL DEFAULT 'en',
    timezone         VARCHAR(60)   NOT NULL DEFAULT 'UTC',

```

```

        settings          JSONB          NOT NULL DEFAULT '{}';
    );

CREATE TABLE user_blocks (
    blocker_id            UUID            NOT NULL REFERENCES users(user_id),
    blocked_id            UUID            NOT NULL REFERENCES users(user_id),
    created_at            TIMESTAMPTZ     NOT NULL DEFAULT now(),
    PRIMARY KEY (blocker_id, blocked_id),
    CONSTRAINT chk_no_self_block CHECK (blocker_id != blocked_id)
);

-- =====
-- CONVERSATIONS
-- =====

CREATE TABLE conversations (
    conv_id              UUID            PRIMARY KEY DEFAULT uuid_generate_v4(),
    conv_type             conversation_type NOT NULL DEFAULT 'direct',
    name                 VARCHAR(500),    -- NULL for direct messages
    description           TEXT,
    avatar_url           VARCHAR(500),
    created_by           UUID            NOT NULL REFERENCES users(user_id),
    is_archived          BOOLEAN         NOT NULL DEFAULT FALSE,
    is_announcement_only BOOLEAN         NOT NULL DEFAULT FALSE, -- only admins can post
    last_message_id      UUID,           -- denormalized, updated on new message
    last_activity_at     TIMESTAMPTZ     NOT NULL DEFAULT now(),
    message_count        BIGINT          NOT NULL DEFAULT 0,
    member_count         INT             NOT NULL DEFAULT 0,
    settings             JSONB          NOT NULL DEFAULT '{}',
    created_at           TIMESTAMPTZ     NOT NULL DEFAULT now(),
    updated_at           TIMESTAMPTZ     NOT NULL DEFAULT now()
);

-- For DMs: ensure only one conversation exists per user pair
CREATE TABLE direct_conversation_map (
    user_a              UUID            NOT NULL REFERENCES users(user_id),
    user_b              UUID            NOT NULL REFERENCES users(user_id),
    conv_id             UUID            NOT NULL REFERENCES conversations(conv_id),
    PRIMARY KEY (user_a, user_b),
    CONSTRAINT chk_user_order CHECK (user_a < user_b) -- enforce canonical order
);

CREATE TABLE conversation_participants (
    conv_id            UUID            NOT NULL REFERENCES conversations(conv_id) ON
DELETE CASCADE,
    user_id            UUID            NOT NULL REFERENCES users(user_id),
    role              member_role     NOT NULL DEFAULT 'member',
    joined_at         TIMESTAMPTZ     NOT NULL DEFAULT now(),
    left_at           TIMESTAMPTZ,
    muted_until       TIMESTAMPTZ,
    is_muted          BOOLEAN         NOT NULL DEFAULT FALSE,
    is_pinned         BOOLEAN         NOT NULL DEFAULT FALSE,
    -- Track per-participant read state

```

```

        last_read_message_id UUID,
        last_read_at    TIMESTAMPTZ,
        unread_count    INT          NOT NULL DEFAULT 0,
        PRIMARY KEY (conv_id, user_id)
    );

-- =====
-- MESSAGES (Core Table – Partitioned)
-- =====
CREATE TABLE messages (
    message_id      UUID          NOT NULL DEFAULT uuid_generate_v4(),
    conv_id         UUID          NOT NULL REFERENCES conversations(conv_id),
    sender_id       UUID          NOT NULL REFERENCES users(user_id),
    message_type    message_type NOT NULL DEFAULT 'text',
    body            TEXT,         -- NULL for media-only messages
    body_format     VARCHAR(20)   NOT NULL DEFAULT 'plain', --
    'plain', 'markdown', 'html'
    parent_id       UUID,         -- for threaded replies (references
    messages(message_id))
    thread_count    INT          NOT NULL DEFAULT 0,
    is_edited       BOOLEAN      NOT NULL DEFAULT FALSE,
    edited_at       TIMESTAMPTZ,
    is_deleted      BOOLEAN      NOT NULL DEFAULT FALSE,
    deleted_at      TIMESTAMPTZ,
    client_msg_id   VARCHAR(100), -- client-side dedup ID
    search_vector    TSVECTOR,
    sent_at         TIMESTAMPTZ   NOT NULL DEFAULT now(),
    PRIMARY KEY (message_id, sent_at)
) PARTITION BY RANGE (sent_at);

-- Daily partitions for recent data (hot)
CREATE TABLE messages_2024_01 PARTITION OF messages
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
-- Weekly or monthly partitions for older data; automate with pg_partman

-- Recent messages hot table (last 7 days – entirely in memory)
-- This is a sub-partition or a separate table depending on workload
CREATE TABLE messages_recent (
    LIKE messages INCLUDING ALL
) PARTITION OF messages FOR VALUES FROM (now() - INTERVAL '7 days') TO ('9999-01-01');

-- =====
-- MESSAGE EDITS HISTORY
-- =====
CREATE TABLE message_edits (
    edit_id         UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    message_id      UUID          NOT NULL,
    conv_id         UUID          NOT NULL,
    body            TEXT          NOT NULL,
    edited_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
    sent_at         TIMESTAMPTZ   NOT NULL -- for partition FK

```

```

);

-- =====
-- ATTACHMENTS
-- =====
CREATE TABLE attachments (
    attachment_id    UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
    message_id       UUID                NOT NULL,
    sent_at          TIMESTAMPTZ         NOT NULL, -- for partition reference
    conv_id          UUID                NOT NULL REFERENCES conversations(conv_id),
    uploader_id      UUID                NOT NULL REFERENCES users(user_id),
    file_name        VARCHAR(500)        NOT NULL,
    file_size_bytes  BIGINT              NOT NULL,
    mime_type        VARCHAR(100)        NOT NULL,
    cdn_url          VARCHAR(1000)       NOT NULL,
    thumbnail_url    VARCHAR(1000),
    width            INT,
    height           INT,
    duration_seconds INT,                -- for audio/video
    is_deleted       BOOLEAN             NOT NULL DEFAULT FALSE,
    created_at       TIMESTAMPTZ         NOT NULL DEFAULT now()
);

-- =====
-- READ RECEIPTS (High Volume)
-- =====
CREATE TABLE read_receipts (
    message_id       UUID                NOT NULL,
    sent_at          TIMESTAMPTZ         NOT NULL,
    user_id          UUID                NOT NULL REFERENCES users(user_id),
    conv_id          UUID                NOT NULL,
    read_at          TIMESTAMPTZ         NOT NULL DEFAULT now(),
    PRIMARY KEY (message_id, user_id, sent_at)
) PARTITION BY RANGE (sent_at);

CREATE TABLE read_receipts_2024_01 PARTITION OF read_receipts
    FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');

-- =====
-- REACTIONS
-- =====
CREATE TABLE reactions (
    reaction_id      UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
    message_id       UUID                NOT NULL,
    sent_at          TIMESTAMPTZ         NOT NULL,
    conv_id          UUID                NOT NULL,
    user_id          UUID                NOT NULL REFERENCES users(user_id),
    emoji            VARCHAR(50)         NOT NULL,
    created_at       TIMESTAMPTZ         NOT NULL DEFAULT now(),
    CONSTRAINT uq_reaction UNIQUE (message_id, user_id, emoji)
);

```

```

-- =====
-- PINNED MESSAGES
-- =====

CREATE TABLE pinned_messages (
    conv_id        UUID            NOT NULL REFERENCES conversations(conv_id),
    message_id     UUID            NOT NULL,
    sent_at        TIMESTAMPTZ     NOT NULL,
    pinned_by      UUID            NOT NULL REFERENCES users(user_id),
    pinned_at       TIMESTAMPTZ     NOT NULL DEFAULT now(),
    PRIMARY KEY (conv_id, message_id)
);

-- =====
-- NOTIFICATIONS
-- =====

CREATE TABLE notifications (
    notification_id UUID            NOT NULL DEFAULT uuid_generate_v4(),
    user_id         UUID            NOT NULL REFERENCES users(user_id),
    notification_type notification_type NOT NULL,
    channel          notification_channel NOT NULL DEFAULT 'in_app',
    title            VARCHAR(300),
    body             VARCHAR(1000),
    data             JSONB,         -- deep link, referenced IDs, etc.
    is_read          BOOLEAN        NOT NULL DEFAULT FALSE,
    read_at          TIMESTAMPTZ,
    created_at       TIMESTAMPTZ     NOT NULL DEFAULT now(),
    PRIMARY KEY (notification_id, created_at)
) PARTITION BY RANGE (created_at);

-- =====
-- REAL-TIME DELIVERY: LISTEN/NOTIFY
-- =====
-- When a message is inserted, notify the conversation channel
CREATE OR REPLACE FUNCTION notify_new_message()
RETURNS TRIGGER AS $$
BEGIN
    -- Channel name: 'conv_' || conv_id (truncated for 63-char limit)
    PERFORM pg_notify(
        'conv_' || replace(NEW.conv_id::text, '-', ''),
        json_build_object(
            'message_id', NEW.message_id,
            'conv_id', NEW.conv_id,
            'sender_id', NEW.sender_id,
            'message_type', NEW.message_type,
            'body', CASE WHEN NEW.is_deleted THEN NULL ELSE LEFT(NEW.body, 200) END,
            'sent_at', NEW.sent_at
        )::text
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_message_notify
  AFTER INSERT ON messages
  FOR EACH ROW EXECUTE FUNCTION notify_new_message();

-- WebSocket server: LISTEN conv_{conv_id_no_dashes}
-- On NOTIFY: push payload to all connected clients in that conversation

-- =====
-- DELIVERY STATUS
-- =====
CREATE TABLE message_delivery_status (
  message_id      UUID          NOT NULL,
  sent_at         TIMESTAMPTZ   NOT NULL,
  recipient_id    UUID          NOT NULL REFERENCES users(user_id),
  status          delivery_status NOT NULL DEFAULT 'queued',
  updated_at      TIMESTAMPTZ   NOT NULL DEFAULT now(),
  PRIMARY KEY (message_id, recipient_id, sent_at)
) PARTITION BY RANGE (sent_at);

-- =====
-- SEARCH INDEX MAINTENANCE
-- =====
CREATE OR REPLACE FUNCTION update_message_search()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.message_type = 'text' AND NOT NEW.is_deleted THEN
    NEW.search_vector := to_tsvector('english', COALESCE(NEW.body, ''));
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_message_search
  BEFORE INSERT OR UPDATE OF body, is_deleted
  ON messages
  FOR EACH ROW EXECUTE FUNCTION update_message_search();

```

ASCII ER Diagram

```

+-----+      +-----+      +-----+
| users | 1-----N | conv_part | N-----1 | conv |
+-----+      +-----+      +-----+
| 1      |      |          |      | 1      |
| N      |      |          |      | N      |
+-----+      +-----+      +-----+
| user_prsnc |  | notifs |      | messages |
+-----+      +-----+      +-----+
|           |      |          |      | 1      |
|           |      |          |      | N      |
+-----+      +-----+      +-----+

```



```

|user_blocks|          |attachments|
+-----+          |
                        +-----+
                        | reactions |
                        +-----+
                        +-----+
                        |read_rcpts |
                        +-----+
                        +-----+
                        |msg_edits  |
                        +-----+

+-----+          +-----+
| conv  |1-----N| pinned_messages |
+-----+          +-----+

+-----+          +-----+
| user_a |          |direct_conv_map  |
+-----+          +-----+
| user_b |          (ensures 1 DM per pair)
+-----+

```

Indexing Strategy

```

-- Users: username search
CREATE INDEX idx_users_username ON users (lower(username));
CREATE INDEX idx_users_username_trgm ON users USING GIN (username gin_trgm_ops);

-- Conversations: user's conversation list (most-used read path)
CREATE INDEX idx_conv_participants_user ON conversation_participants
  (user_id, last_activity_at DESC) -- actually on conversations via join; index
  join column
  WHERE left_at IS NULL;
CREATE INDEX idx_conv_activity ON conversations (last_activity_at DESC);
CREATE INDEX idx_conv_type ON conversations (conv_type);

-- Messages: conversation timeline (most critical read path)
CREATE INDEX idx_messages_conv_time ON messages (conv_id, sent_at DESC);

-- Messages: full-text search (per conversation or global)
CREATE INDEX idx_messages_search ON messages USING GIN (search_vector)
  WHERE NOT is_deleted;

-- Messages: thread replies
CREATE INDEX idx_messages_parent ON messages (parent_id, sent_at ASC)
  WHERE parent_id IS NOT NULL;

-- Messages: by sender (for user's sent messages view)
CREATE INDEX idx_messages_sender ON messages (sender_id, sent_at DESC);

```

```

-- Read receipts: unread count per user per conv (hot path)
CREATE INDEX idx_read_receipts_user ON read_receipts (user_id, conv_id, read_at
DESC);

-- Reactions: per message (emoji summary)
CREATE INDEX idx_reactions_message ON reactions (message_id, emoji);

-- Notifications: user inbox
CREATE INDEX idx_notifications_user ON notifications
  (user_id, created_at DESC)
  WHERE is_read = FALSE;

-- Presence: updated constantly
CREATE INDEX idx_presence_updated ON user_presence (updated_at DESC);

-- Direct conversation map: fast DM lookup
-- Primary key index serves (user_a, user_b) lookups
-- Add reverse index for user_b queries:
CREATE INDEX idx_dcm_user_b ON direct_conversation_map (user_b, user_a);

```

Query Patterns

Q1 — Load User's Inbox (Conversation List)

```

SELECT
  c.conv_id, c.conv_type, c.name, c.avatar_url,
  c.last_activity_at,
  cp.unread_count, cp.is_muted, cp.is_pinned,
  -- Last message preview
  m.body AS last_message_body,
  m.message_type AS last_message_type,
  u2.display_name AS last_sender_name,
  m.sent_at AS last_message_at,
  -- For DMs: the other user
  CASE WHEN c.conv_type = 'direct' THEN
    (SELECT u3.display_name
     FROM direct_conversation_map dcm
     JOIN users u3 ON u3.user_id = CASE WHEN dcm.user_a = $1 THEN dcm.user_b
    ELSE dcm.user_a END
     WHERE dcm.conv_id = c.conv_id)
  END AS dm_counterpart_name
FROM conversation_participants cp
JOIN conversations c ON c.conv_id = cp.conv_id
LEFT JOIN messages m ON m.message_id = c.last_message_id
  AND m.sent_at >= now() - INTERVAL '30 days' -- partition pruning hint
LEFT JOIN users u2 ON u2.user_id = m.sender_id
WHERE cp.user_id = $1
  AND cp.left_at IS NULL

```

```
ORDER BY cp.is_pinned DESC, c.last_activity_at DESC
LIMIT 50;
```

Q2 — Load Conversation Messages (Timeline)

```
SELECT
    m.message_id, m.conv_id, m.sender_id,
    m.message_type, m.body, m.body_format,
    m.parent_id, m.thread_count,
    m.is_edited, m.is_deleted, m.sent_at,
    u.display_name, u.avatar_url,
    json_agg(DISTINCT jsonb_build_object('emoji', r.emoji, 'count',
        (SELECT COUNT(*) FROM reactions r2 WHERE r2.message_id = m.message_id AND
         r2.emoji = r.emoji)
    )) FILTER (WHERE r.emoji IS NOT NULL) AS reactions,
    json_agg(DISTINCT jsonb_build_object(
        'url', a.cdn_url, 'type', a.mime_type, 'name', a.file_name
    )) FILTER (WHERE a.attachment_id IS NOT NULL) AS attachments
FROM messages m
JOIN users u ON u.user_id = m.sender_id
LEFT JOIN reactions r ON r.message_id = m.message_id
LEFT JOIN attachments a ON a.message_id = m.message_id
WHERE m.conv_id = $1
    AND m.sent_at < $before_time      -- cursor-based pagination
    AND m.sent_at >= $after_time     -- partition pruning
    AND m.parent_id IS NULL          -- top-level messages only
GROUP BY m.message_id, u.user_id
ORDER BY m.sent_at DESC
LIMIT 50;
```

Q3 — Send Message (with NOTIFY)

```
BEGIN;

-- Insert message (triggers pg_notify automatically)
INSERT INTO messages (
    message_id, conv_id, sender_id, message_type, body, client_msg_id, sent_at
) VALUES (
    uuid_generate_v4(), $conv_id, $sender_id, $type, $body, $client_msg_id, now()
)
ON CONFLICT DO NOTHING -- idempotent via client_msg_id
RETURNING message_id, sent_at;

-- Update conversation last activity
UPDATE conversations
SET last_message_id = $message_id,
    last_activity_at = now(),
    message_count = message_count + 1,
    updated_at = now()
```

```

WHERE conv_id = $conv_id;

-- Increment unread count for all participants except sender
UPDATE conversation_participants
SET unread_count = unread_count + 1
WHERE conv_id = $conv_id
    AND user_id != $sender_id
    AND left_at IS NULL
    AND NOT is_muted;

COMMIT;

```

Q4 — Mark Messages as Read

```

BEGIN;

-- Bulk insert read receipts
INSERT INTO read_receipts (message_id, sent_at, user_id, conv_id, read_at)
SELECT m.message_id, m.sent_at, $user_id, $conv_id, now()
FROM messages m
WHERE m.conv_id = $conv_id
    AND m.sent_at > $last_read_at -- only newly read messages
    AND m.sent_at >= now() - INTERVAL '30 days' -- partition pruning
ON CONFLICT DO NOTHING;

-- Reset unread counter
UPDATE conversation_participants
SET unread_count = 0,
    last_read_message_id = $latest_message_id,
    last_read_at = now()
WHERE conv_id = $conv_id AND user_id = $user_id;

COMMIT;

```

Q5 — Message Search in Conversation

```

SELECT
    m.message_id, m.conv_id, m.sender_id,
    m.body, m.sent_at,
    ts_headline('english', m.body,
        to_tsquery('english', $query),
        'MaxWords=20, MinWords=5, ShortWord=3, HighlightAll=false'
    ) AS highlighted,
    u.display_name, u.avatar_url,
    ts_rank(m.search_vector, to_tsquery('english', $query)) AS rank
FROM messages m
JOIN users u ON u.user_id = m.sender_id
WHERE m.conv_id = $1
    AND m.search_vector @@ to_tsquery('english', $query)

```

```
    AND NOT m.is_deleted
ORDER BY rank DESC, m.sent_at DESC
LIMIT 20;
```

Q6 — Get Typing Indicators (via LISTEN/NOTIFY)

```
-- Application layer: when user starts typing:
SELECT pg_notify(
    'typing_' || replace($conv_id::text, '-', ''),
    json_build_object(
        'user_id', $user_id,
        'display_name', $display_name,
        'action', 'typing'      -- or 'stopped_typing'
    )::text
);
-- WebSocket server subscribes to 'typing_*' channels
-- No DB rows written for typing indicators
```

Q7 — Conversation Members List

```
SELECT
    u.user_id, u.display_name, u.avatar_url,
    cp.role, cp.joined_at, cp.is_muted,
    up.status AS presence,
    up.last_seen_at
FROM conversation_participants cp
JOIN users u ON u.user_id = cp.user_id
LEFT JOIN user_presence up ON up.user_id = cp.user_id
WHERE cp.conv_id = $1
    AND cp.left_at IS NULL
ORDER BY
    CASE cp.role WHEN 'owner' THEN 1 WHEN 'admin' THEN 2 ELSE 3 END,
    u.display_name
LIMIT 200;
```

Q8 — Thread: Load Replies to a Message

```
SELECT
    m.message_id, m.sender_id, m.body, m.sent_at,
    m.is_edited, m.is_deleted,
    u.display_name, u.avatar_url
FROM messages m
JOIN users u ON u.user_id = m.sender_id
WHERE m.parent_id = $1
    AND m.sent_at >= $thread_start_time  -- partition pruning
ORDER BY m.sent_at ASC
LIMIT 100;
```

Q9 — Notification Inbox

```
SELECT
    n.notification_id, n.notification_type,
    n.title, n.body, n.data,
    n.is_read, n.created_at
FROM notifications n
WHERE n.user_id = $1
    AND n.created_at >= now() - INTERVAL '30 days'
ORDER BY n.created_at DESC
LIMIT 50 OFFSET $2;
```

Q10 — Find or Create DM Conversation

```
-- Efficient: canonical (user_a < user_b) order prevents race condition
WITH dm_lookup AS (
    SELECT conv_id
    FROM direct_conversation_map
    WHERE (user_a = LEAST($user1, $user2) AND user_b = GREATEST($user1, $user2))
),
new_conv AS (
    INSERT INTO conversations (conv_id, conv_type, created_by)
    SELECT uuid_generate_v4(), 'direct', $user1
    WHERE NOT EXISTS (SELECT 1 FROM dm_lookup)
    RETURNING conv_id
),
insert_map AS (
    INSERT INTO direct_conversation_map (user_a, user_b, conv_id)
    SELECT LEAST($user1, $user2), GREATEST($user1, $user2), conv_id
    FROM new_conv
    ON CONFLICT DO NOTHING
    RETURNING conv_id
),
add_participants AS (
    INSERT INTO conversation_participants (conv_id, user_id)
    SELECT conv_id, unnest(ARRAY[$user1, $user2])
    FROM new_conv
    ON CONFLICT DO NOTHING
)
SELECT COALESCE(
    (SELECT conv_id FROM dm_lookup),
    (SELECT conv_id FROM new_conv)
) AS conv_id;
```

Scaling Strategy

Phase 1: Single Cluster (0–100M users)

- PostgreSQL with 2 read replicas

- LISTEN/NOTIFY via WebSocket servers connected to primary
- Redis for typing indicators and presence (never hit Postgres)
- Messages partitioned monthly
- Attachments stored in S3 (only metadata in DB)

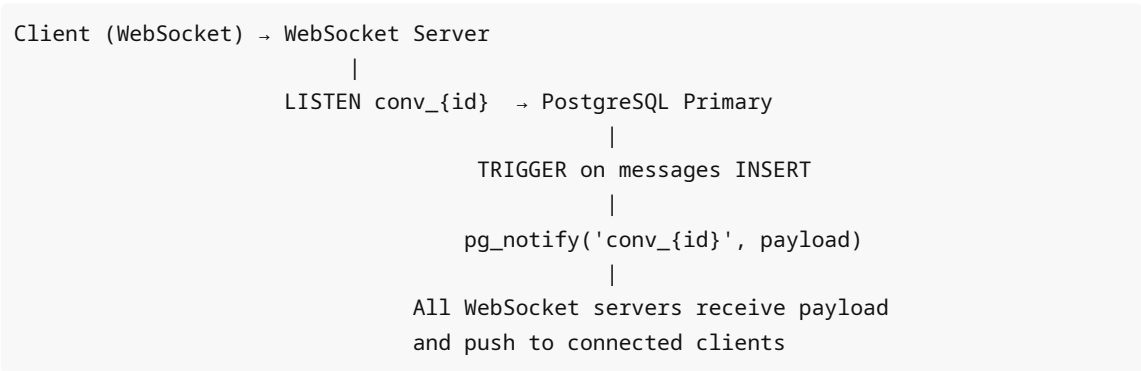
Phase 2: Sharding Read Paths (100M–500M users)

- Messages table sharded by conv_id % N (keeps all messages of a conversation on the same shard)
- Each shard has its own read replicas
- Conversation metadata (conversations, conversation_participants) on a dedicated metadata cluster
- LISTEN/NOTIFY: per-shard — WebSocket servers connect to the shard that owns the conversation
- Redis cluster for presence and typing (globally distributed, eventual consistency OK)

Phase 3: Hybrid Storage (500M+ users)

- Hot messages (last 90 days): PostgreSQL — full feature set (search, reactions, edits)
- Warm messages (90 days – 2 years): Apache Cassandra — time-series optimized, wide rows per conversation
- Cold messages (2+ years): Object storage (S3) with metadata in PostgreSQL
- Message routing service: transparent to clients, routes requests to appropriate store

LISTEN/NOTIFY Architecture



Limitation: LISTEN/NOTIFY does not work across replicas. All LISTEN connections must go to the primary. At 100M DAU with 5 conversations open = 500M LISTEN channels — exceeds single PostgreSQL. At that scale, move to Redis Pub/Sub or Kafka.

Tradeoffs

Decision	Chosen	Alternative	Why
Daily message partitions	Daily	Monthly	Hot recent data in 1-2 partitions; hot data fits in buffer cache
UNLOGGED presence table	UNLOGGED	WAL-logged	Presence is ephemeral; losing it on crash is safe; 5x write throughput
Denormalized unread_count	Counter on conv_participants	COUNT(*) at query time	COUNT(*) on billions of read_receipts per page load is unacceptable

last_message_id on conversations	Denormalized	Subquery	Single column lookup vs correlated subquery on every inbox load
LISTEN/NOTIFY for real-time	pg_notify	Polling	Push model; eliminates 1000s of polling queries per second
Canonical DM (user_a < user_b)	Ordered pair	Separate lookup	Prevents duplicate DM conversations between same pair

Interview Discussion Points

- How does real-time message delivery work?** PostgreSQL LISTEN/NOTIFY: the WebSocket server issues `LISTEN conv_{id}` for every open conversation. When any client sends a message, the INSERT trigger fires `pg_notify('conv_{id}', payload)`. PostgreSQL multicasts the payload to all listening connections, which push it to their connected WebSocket clients. No polling.
- How do you handle message ordering?** Messages use `sent_at TIMESTAMPTZ`. Since multiple servers can insert simultaneously, `sent_at` alone is not unique — the PRIMARY KEY is `(message_id, sent_at)`. Client-side, messages are ordered by `sent_at, message_id` for a stable, deterministic sort.
- How do you prevent the "thundering herd" on group messages?** For a group with 1000 members, one message triggers 1000 `UPDATE conversation_participants SET unread_count = unread_count + 1`. Use a batch update with `user_id IN (subquery)` or denormalize differently — push notifications via Kafka fanout service; only update `unread_count` lazily.
- How does cursor-based pagination work for messages?** The client stores the `sent_at` of the oldest loaded message. The next page query is `WHERE sent_at < $cursor ORDER BY sent_at DESC LIMIT 50`. This avoids the OFFSET problem (skipped rows when new messages arrive) and enables partition pruning.

Common Interview Follow-ups

Q: How do you handle a user deleting their account? A: Soft delete: set `users.display_name = 'Deleted User'`, nullify PII fields, `users.is_deleted = TRUE`. Messages authored by the user are soft-deleted (`body = NULL, is_deleted = TRUE`) but the `message_id` remains for read receipt references. This prevents gaps in conversation timelines.

Q: How do you implement end-to-end encryption? A: The DB stores only ciphertext for `messages.body`. The encryption key is derived from participants' public keys (Signal Protocol / double ratchet). The PostgreSQL schema is unchanged — `body TEXT` simply holds base64-encoded ciphertext. Server never sees plaintext. Full-text search is unavailable for E2E encrypted messages.

Q: How would you scale to 1 billion concurrent users? A: PostgreSQL handles metadata and recent messages per-region. LISTEN/NOTIFY replaced by Redis Pub/Sub (horizontally scalable). Message storage moved to Cassandra (wide row per conversation, time-ordered). WebSocket servers are stateless, horizontally scaled, backed by Redis for session routing.

Performance Considerations

- **pg_notify payload limit:** 8000 bytes per notification. Keep payload minimal (IDs, not full message body). Recipients fetch full message on receipt.
 - **Partition pruning on messages:** Always include `sent_at >=` bound in message queries. Without it, PostgreSQL scans all partitions.
 - **UNLOGGED presence table:** No WAL = 5–10x faster writes. On crash, presence table is empty — acceptable since clients heartbeat every 30s and repopulate it.
 - **HOT updates on conversation_participants:** `unread_count` is incremented on every message. Use `fillfactor=70` to enable Heap-Only Tuple updates.
 - **Reaction aggregation:** Don't COUNT(*) reactions per emoji per message at query time. Cache aggregates in Redis, invalidate on reaction change.
 - **Full-text search at scale:** PostgreSQL GIN search is fine for per-conversation search. For cross-conversation global search, use Elasticsearch with a PostgreSQL logical replication consumer.
-

Cross-References

- See `21_System_Design/06_chat_application_design.md` for full system architecture
- See `02_netflix_streaming_platform.md` for LISTEN/NOTIFY presence patterns
- See `21_System_Design/08_system_design_framework.md` for interview framework
- PostgreSQL LISTEN/NOTIFY: Chapter 10 of this guide
- Table partitioning: Chapter 5 of this guide
- Full-text search: Chapter 9 of this guide